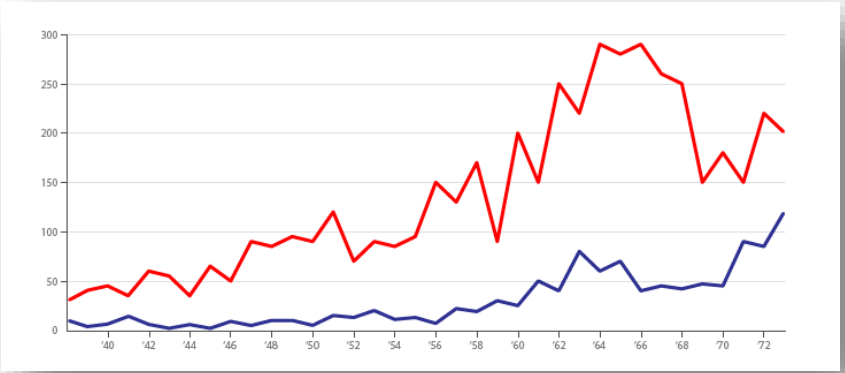
P_FUN



Kiril Bormin
ETML
24.08.2026 / 30.10.2026
Xavier Carrel

Table des matières

1 SPÉCIFICATIONS 3
1.1 TITRE 3
1.2 DESCRIPTION 3
1.3 DOMAINE ET SOURCE DE DONNÉES 3
1.4 OBJECTIFS 3
2 PLANIFICATION/GESTION PROJET 3
2.1 PLANIFICATION INITIALE 3
3 ANALYSE FONCTIONNELLE 4
3.1 #1 — AJOUT D'UNE NOUVELLE SERIE DEPUIS UN FICHIER 4
3.2 #2 — MASQUER UNE SÉRIE 4
3.3 #3 — INTERACTION AVEC LE GRAPHIQUE 5
3.4 #4 — LABELS DES GRAPHIQUES 6
3.5 #5 — AJOUT DE JEU DE DONNÉES DEPUIS UNE API 6
3.6 #6 — SAISIE MANUELLE D'UN NOUVEAU POINT 7
3.7 #7 — EXPORTATION EN PNG 8
4 POINTS TECHNIQUES 9
4.1 PARSER DES CSV 9
4.2 FUSION DES SÉRIES 9
4.3 FORMATAGE DES GRANDS NOMBRES 9
5 TESTS 9
5.1 STRATÉGIE ET EXÉCUTION 9
5.2 SERVICE PARSER (PARSERTESTS.CS) : 10
5.3 SERVICE DE FUSION (SERIESMERGETESTS.CS) : 10
5.4 SERVICE DE FORMATAGE (NUMBERTRANSFORMATORTESTS.CS) : 10
6 USAGE DE L'IA 10
7 CONCLUSION 12
7.1 BILAN DE LA PLANIFICATION 12
7.2 BILAN DES USERS STORIES 12
7.3 BILAN DES PROBLÈMES 13
7.4 BILAN DU PROJET 13
8 WEBOGRAPHIE 13

1 SPÉCIFICATIONS

1.1 Titre

PlotThoseLines

1.2 Description

Le projet est une application C# / Windows Forms, qui permet d'afficher et de comparer facilement des courbes de données (séries de données dans le temps). L'utilisateur peut utiliser 2 sources de données : json et csv.

1.3 Domaine et source de données

Le domaine choisi est la **finance**, plus précisément l'analyse boursière des plus grandes entreprises mondiales par capitalisation boursière (Big Tech).

L'application exploite 5(ou plus) séries temporelles cohérentes (500+ valeurs chacune) :

- Nvidia
- Apple
- Microsoft
- Google
- Amazon
- …

Ces 5 entreprises dominent les indices mondiaux.
Les afficher ensemble permet de comparer la croissance entre les concurrents.

Sources des données – Fichiers .csv ou json exporté depuis Yahoo Finance et données en direct via une API financière.

1.4 Objectifs

Objectif produit : Créer une application simple et rapide pour importer des données, les enregistrer en stockage local pour travailler hors ligne, et afficher plusieurs courbes en même temps à l'aide de la librairie ScottPlot.

Objectif pédagogique : Utiliser les pratiques de la programmation fonctionnelle, apprendre à utiliser des librairies en C#, améliorer la gestion d'un projet avec la méthodologie agile.

2 PLANIFICATION/GESTION PROJET

2.1 Planification initiale

Semaine 2	Création et amélioration des US
Semaine 3	Création de code de base en C#
Semaine 4	Intégration de la librairie ScottPlot
Semaine 5	Intégration de la librairie ScottPlot
Semaine 6	Ajout de fonctionnalité quand plusieurs graphiques sont affichés simultanément, ajout de légende
Semaine 7	Tests unitaires et retouches

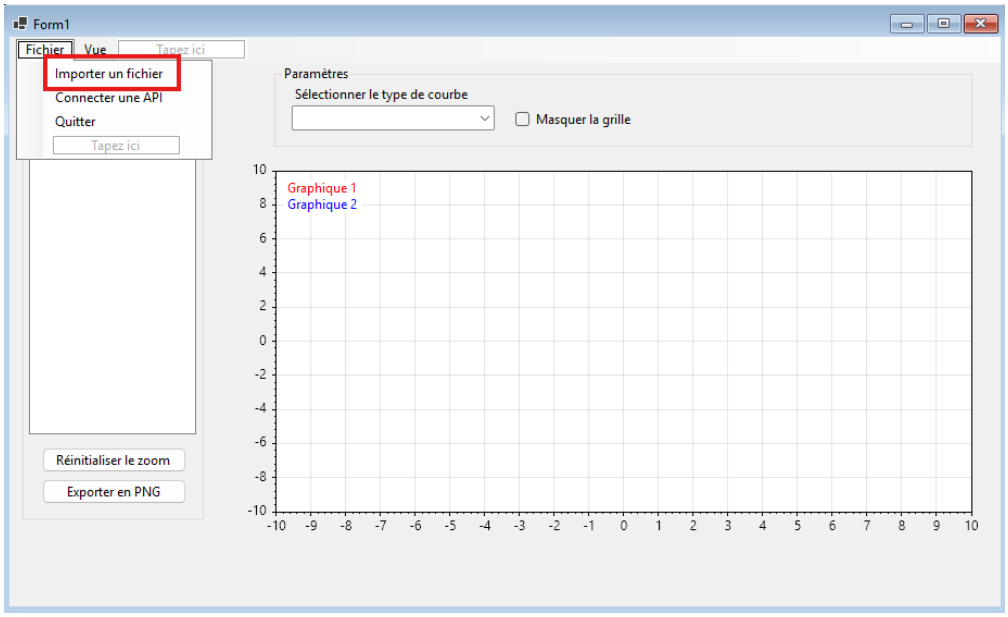
Semaine 8 (après les vacances)	Améliorations et rendu du projet

3 ANALYSE FONCTIONNELLE

3.1 #1 — Ajout d'une nouvelle serie depuis un fichier

En tant qu'utilisateur,
Je veux importer un fichier de données local (.csv, .json),
Afin de charger et stocker de nouvelles séries temporelles pour une utilisation hors ligne.

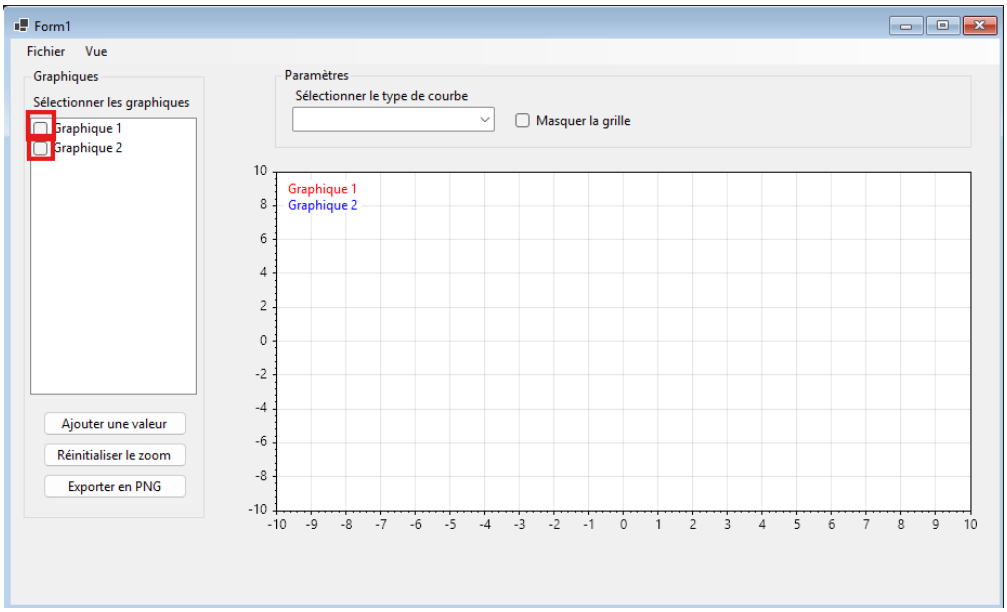
- Accès : L'option est accessible depuis le menu Fichier > Importer un fichier.
- Sélection : Une boîte de dialogue d'exploration de fichiers s'ouvre avec un filtre prédéfini sur les extensions .csv et .json.
- Succès : Le fichier est lu et les données sont enregistrées dans le stockage local -> La nouvelle série s'ajoute automatiquement à la liste des graphiques cochables à gauche.
- Cas d'erreur : Si le fichier est corrompu, vide ou mal structuré, l'application ne plante pas - > une boîte de dialogue s'affiche au centre avec un message explicite.



3.2 #2 — Masquer une série

En tant qu'utilisateur,
Je veux pouvoir masquer une serie de la vue active,
Afin de pouvoir gérer les graphiques que je veux apercevoir.

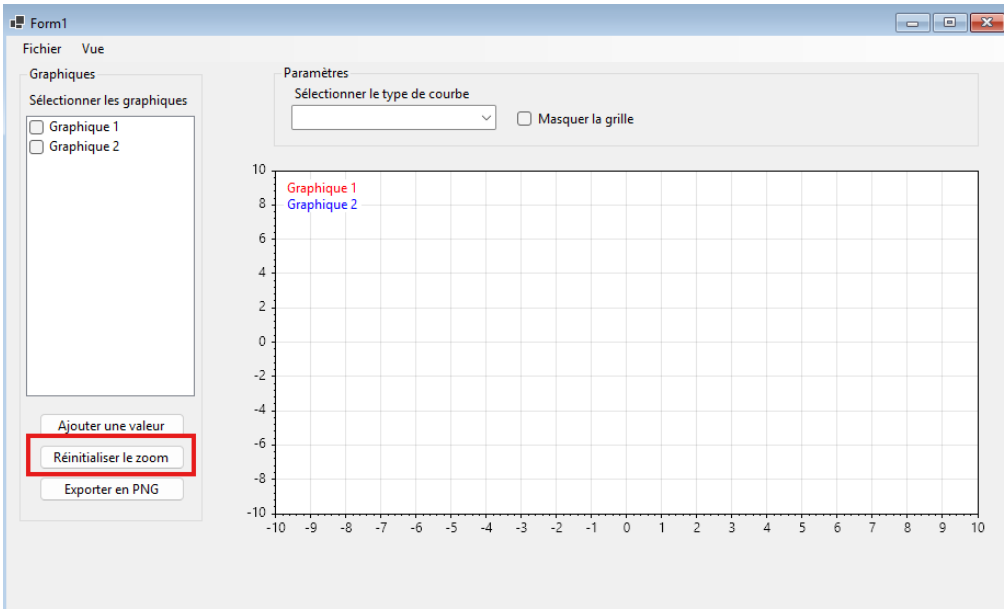
- Une liste cochable de suppression retire une courbe du graphique immédiatement.



3.3 #3 — Interaction avec le graphique

En tant qu'utilisateur,
Je veux pouvoir interagir avec le graphique, en zoomant et en le déplaçant dans tous les sens.
Afin de pouvoir mieux analyser les données

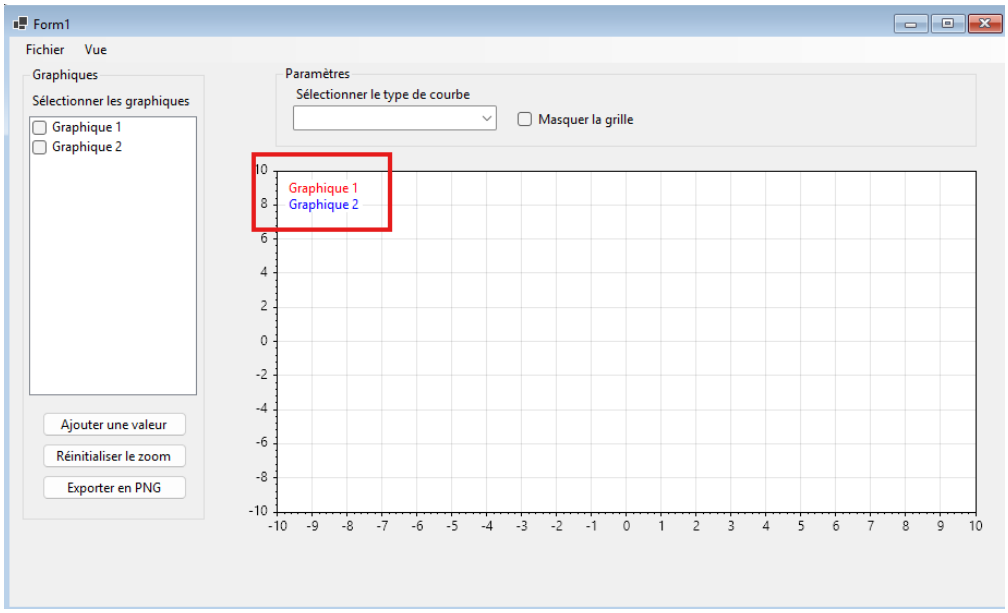
- Actionner la molette de la souris sur le graphique -> L'échelle temporelle s'ajuste fluidement (zoom avant/arrière) autour du curseur.
- Un clique-glisser défile le long des axes
- Un bouton qui permet de réinitialiser le zoom est présent et fonctionnel



3.4 #4 — Labels des graphiques

En tant qu'utilisateur,
Je veux pouvoir identifier chaque graphique à l'aide de sa couleur et de la légende dans un coin de la fenêtre,
Afin de ne pas confondre plusieurs graphiques

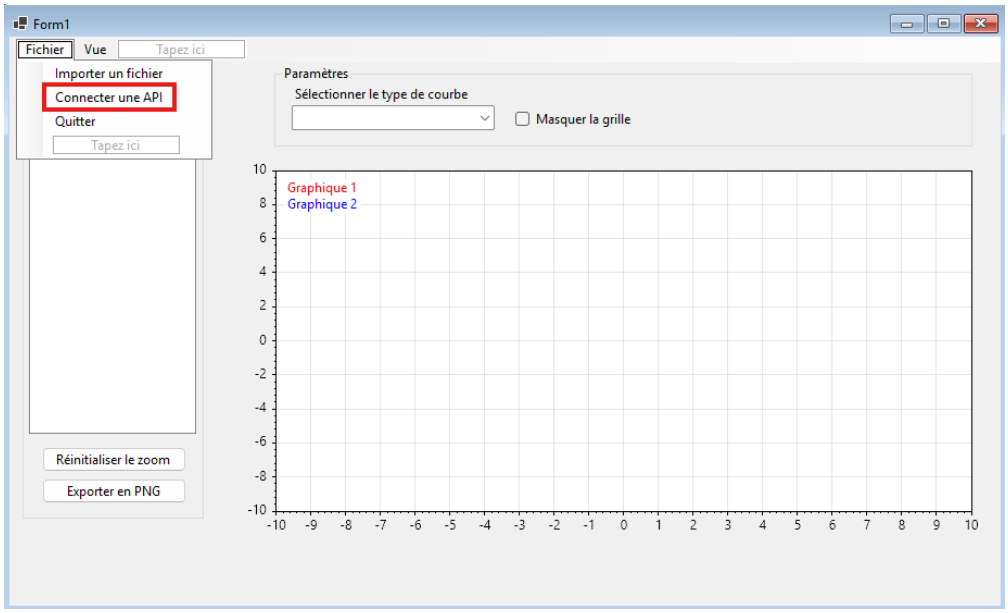
- Si plusieurs séries sont affichées en même temps, chaque courbe est tracée avec une couleur propre et contrastée
- Afficher ou masquer une série -> La légende actualise instantanément le nom et la pastille de couleur correspondante.



3.5 #5 — Ajout de jeu de données depuis une API

En tant qu'utilisateur,
Je veux pouvoir récupérer et intégrer un jeu de données directement depuis une API externe.
Afin de charger des données distantes à jour et les stocker localement pour une utilisation hors ligne.

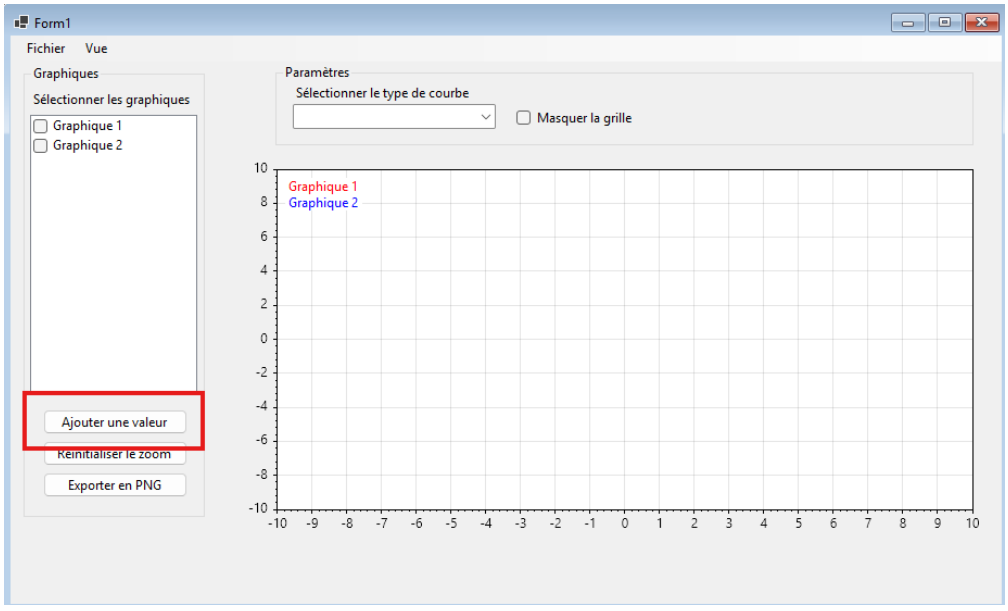
- Sur la page principale, dans le menu « Fichier », je vois l'option appelée « Importer depuis une API ».
- En cliquant sur cette option, une fenêtre s'affiche pour saisir l'URL de l'API (point de terminaison) et les paramètres de requête éventuels (ex. clé d'API, dates).
  - Après validation et récupération des données, la (les) série(s) est enregistrée dans le stockage local et devient disponible dans la liste des graphiques.
  - Si l'URL est invalide, la connexion réseau échoue ou le format des données reçues est corrompu/incompatible, une fenêtre avec un message d'erreur explicite s'affiche au centre.

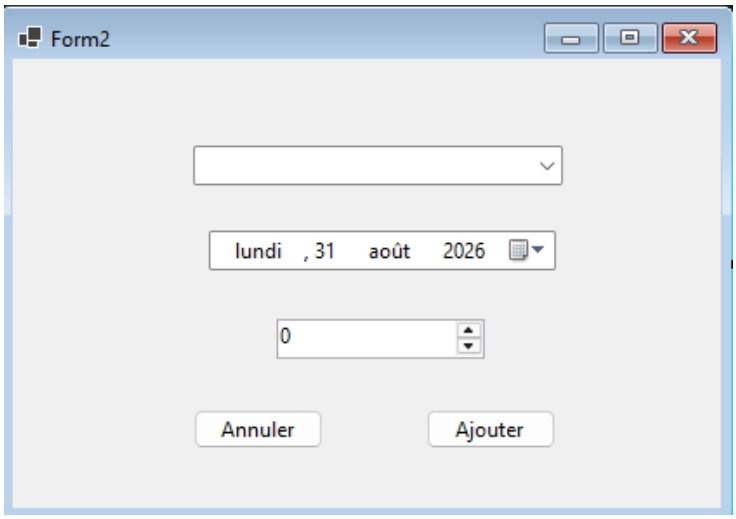


3.6 #6 — Saisie manuelle d'un nouveau point

En tant qu'utilisateur,
Je veux pouvoir saisir manuellement un point (date/heure et valeur) dans une série déjà enregistrée,
Afin de compléter ou corriger mes données locales hors ligne.

- Un formulaire simple permet de sélectionner la série, d'entrer la date et la valeur numérique.
- Le graphique et le fichier local se mettent à jour automatiquement.

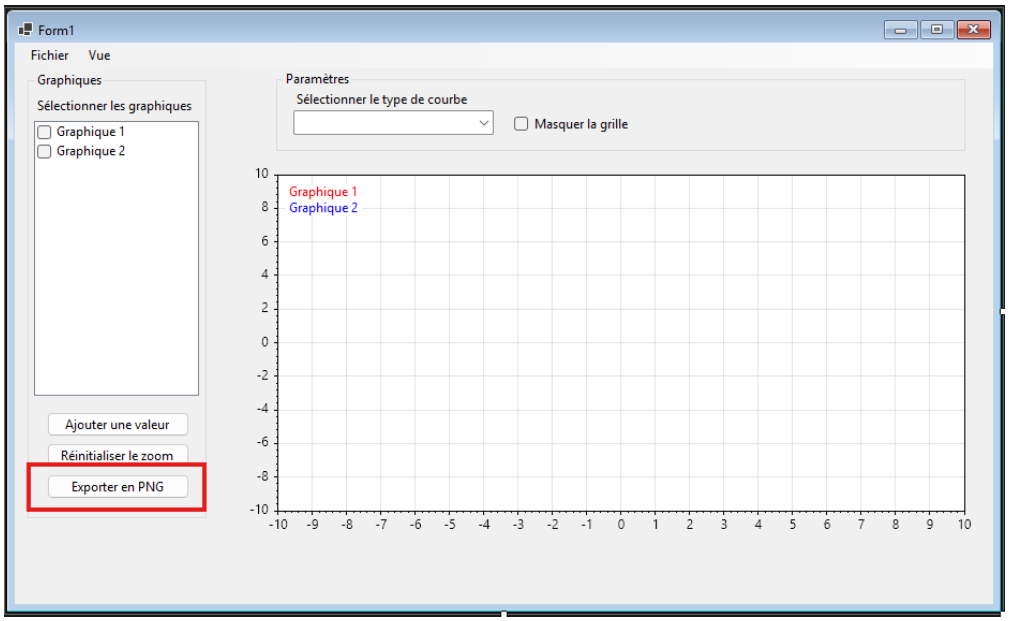




3.7 #7 — Exportation en PNG

En tant qu'utilisateur,
Je veux pouvoir exporter le graphique affiché sous forme d'image au format .png,
Afin d'enregistrer mes visualisations pour les partager ou les insérer dans un rapport

- Sur la page principale, je clique sur le bouton « Exporter en PNG ».
- Une fenêtre de sauvegarde de fichier s'affiche pour choisir le nom du fichier et l'emplacement sur l'ordinateur.
- Après validation, l'image exacte du graphique affiché à l'écran est enregistrée.
- Si l'enregistrement échoue un message d'erreur explicite s'affiche au centre.



4 POINTS TECHNIQUES

4.1 Parser des csv

Pour extraire les données des fichiers CSV sans alourdir l'interface, la logique a été déplacée dans une classe dédiée Parser.cs

Ce parseur utilise des méthodes LINQ à la place de boucles foreach. Il recherche automatiquement les colonnes de date et de valeur grâce à une liste d'alias possibles Si une colonne manque ou si le fichier est vide, une exception est levée pour informer l'utilisateur sans faire planter le programme.

4.2 Fusion des séries

Également pour la logique de fusion d'une ancienne série avec une nouvelle, j'ai créé une classe séparée SeriesMerge.cs.

La méthode identifie la période couverte par la nouvelle série, il filtre la série existante pour retirer les anciens points de cette période et les remplacer par les nouveaux.

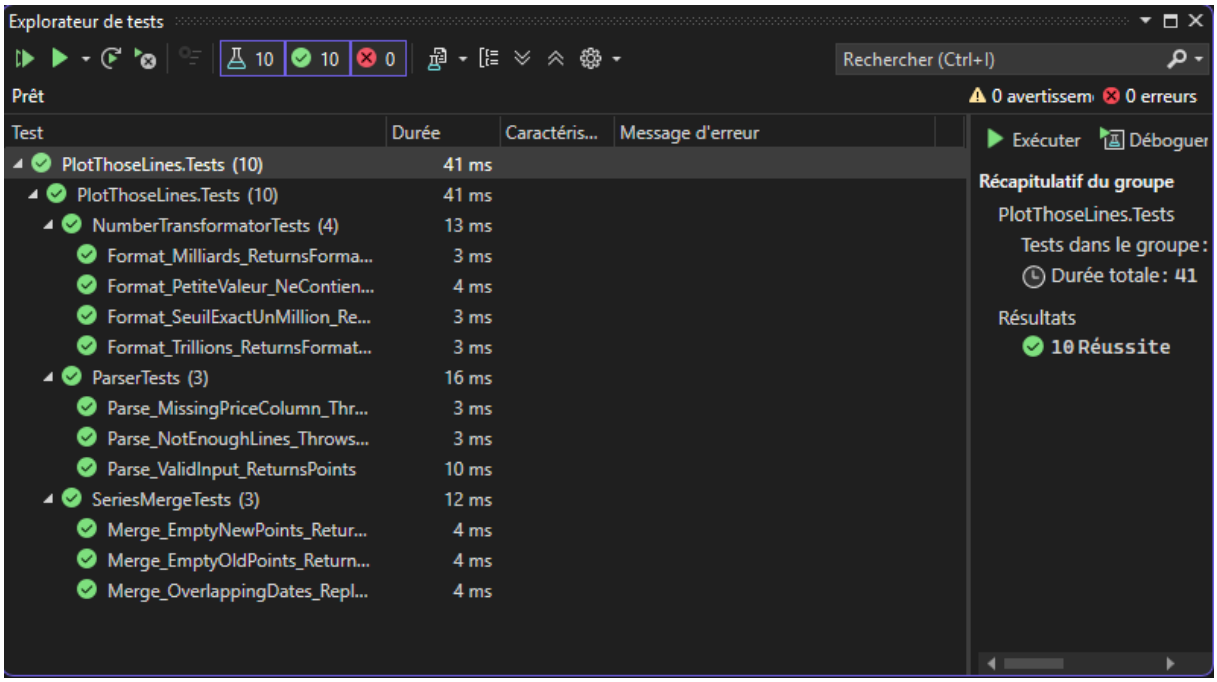
4.3 Formatage des grands nombres

Comme la capitalisation boursière des grandes entreprises dépasse souvent le milliard ou le billion de dollars, les nombres sur l'axe Y étaient trop longs à lire (par ex. : 3 000 000 000 000). J'ai donc créé la classe NumberFormatter pour transformer ces grands nombres en texte plus court avec des lettres : M pour les millions, Md pour les milliards et T pour les trillions. Cette méthode est reliée directement à l'axe gauche du graphique ScottPlot grâce à l'option LabelFormatter.

5 TESTS

5.1 Stratégie et exécution

Les règles métier ont été extraites de MainForm.cs vers des classes de tests avec MSTest dans le projet PlotThoseLines.Tests. L'ensemble de la suite compte 10 tests unitaires, tous validés avec succès.



5.2 Service Parser (ParserTests.cs) :

- Validation de l'extraction d'un point valide avec colonnes reconnues par alias.
- Levée d'une InvalidDataException si le fichier ne contient aucune ligne de données.
- Levée d'une InvalidDataException si la colonne de prix/capitalisation est absente.

5.3 Service de fusion (SeriesMergeTests.cs) :

- Remplacement correct des anciens points par les nouveaux sur la période de dates commune (chevauchement).
- Comportement normal et renvoi direct des nouveaux points si l'ancienne série est vide.
- Préservation des points existants sans erreur si le nouvel import est vide.

5.4 Service de formatage (NumberTransformatorTests.cs) :

- Trillions formatés avec le suffixe " T".
- Milliards formatés avec le suffixe " Md".
- Seuil exact d'un million formaté avec "1 M".
- Valeurs inférieures au million conservées au format numérique standard sans suffixe.

6 USAGE DE L'IA

Pour tester l'application avec beaucoup de données (entre 4 000 et 5 000 valeurs par entreprise), je n'ai pas trouvé de site gratuit qui donne directement l'historique de la capitalisation boursière en fichier CSV. Sur Yahoo Finance, on peut seulement avoir le prix d'une action mais pas la capitalisation totale. J'ai donc demandé à l'IA de me créer un script en Node.js en utilisant la librairie yahoo-finance2. Ce script récupère d'abord le nombre total d'actions de l'entreprise puis il télécharge le prix de l'action chaque jour sur 20 ans (dans mon cas). Enfin, il multiplie le prix du jour par le nombre d'actions pour calculer la capitalisation boursière et enregistre le résultat dans un fichier CSV.

J'ai aussi utilisé l'IA pour générer trois fichiers avec des données fictives entre 2025 et 2035 afin de tester la fusion des courbes.

Prompts :

- « comment trouver un csv avec les données de capitalisation des plus grands companies du monde ? »
- « crée moi un script en python qui permet de récupérer l'historique sur des années avec la fréquance une fois par jour pour les 20 années en nodejs»

Code :

```
const pkg = require('yahoo-finance2');
const YahooFinance = pkg.YahooFinance || pkg.default || pkg;
const yahooFinance = new YahooFinance({ suppressNotices: ['yahooSurvey', 'ripHistorical'] });
const fs = require('fs');


async function exportDailyMarketCap(ticker, years = 10) {
  try {
    // 1. Récupération des actions en circulation
    const quote = await yahooFinance.quote(ticker);
```

```
    const sharesOutstanding = quote.sharesOutstanding;

    if (!sharesOutstanding) {
      throw new Error("Impossible de récupérer le nombre d'actions en circulation.");
    }

    // 2. Plage de dates
    const endDate = new Date();
    const startDate = new Date();
    startDate.setFullYear(endDate.getFullYear() - years);

    // 3. Récupération des cours au format quotidien ('1d')
    const result = await yahooFinance.chart(ticker, {
      period1: startDate,
      period2: endDate,
      interval: '1d'
    });

    const quotes = result.quotes || [];

    // 4. Construction du CSV
    const header = 'Date,Close,Shares_Outstanding,Market_Cap\n';
    const rows = quotes
      .filter(row => row.close != null)
      .map(row => {
        const date = new Date(row.date).toISOString().split('T')[0];
        const close = row.close.toFixed(2);
        const marketCap = (row.close * sharesOutstanding).toFixed(0);
        return `${date},${close},${sharesOutstanding},${marketCap}`;
      })
      .join('\n');

    // 5. Écriture du fichier CSV
    const fileName = `${ticker}_daily_market_cap_${years}y.csv`;
    fs.writeFileSync(fileName, header + rows, 'utf-8');
    console.log(`Succès : ${quotes.length} jours enregistrés dans ${fileName}`);
  } catch (error) {
    console.error('Erreur :', error.message);
  }
}

// Exemple pour Apple sur 20 ans
exportDailyMarketCap('AAPL', 10);
```

De l'IA a été également utilisée dans plusieurs endroits dans le code afin de m'aider à comprendre certains concepts ou me conseiller une solution adaptée tout en l'expliquant.

7 CONCLUSION

7.1 Bilan de la planification

Ma planification initiale était surestimé, j'ai laissé 2 semaines pour l'intégration de Scottplot, mais apparemment c'est beaucoup plus simple que je pensais, et a pris beaucoup moins de temps.
Dans la planification initiale, je n'ai pas pris en compte que le lundi de la semaine 5 est congé, et qu'il ne fallait rien planifier pour ce jour-là.

7.2 Bilan des users stories

Dans GitHub Projects, j'ai défini plusieurs User Stories pour structurer le développement de l'application. La majorité d'entre elles ont été faites avec succès, tandis que certaines ont dû être abandonnées ou adaptées, principalement en raison de contraintes de temps et de pertinence technique.

User Stories réalisées :

US #1 — Ajout d'une nouvelle série depuis un fichier (partie CSV) -> L'utilisateur peut importer un fichier .csv local depuis le menu. Les colonnes de date et de valeur sont automatiquement identifiées, et les données sont enregistrées en local puis tracées sur le graphique. En cas de fichier vide ou corrompu, un message d'erreur clair est affiché.
US #2 — Masquer une série -> L'interface propose une liste à cocher qui permet d'afficher ou de masquer instantanément n'importe quelle courbe sans recharger les données.
US #3 — Interaction avec le graphique -> L'utilisateur peut zoomer fluidement avec la molette de la souris, faire défiler les axes en cliquer-glisser, et réinitialiser la vue d'ensemble en un clic grâce au bouton « Réinit. le zoom ».

US #4 — Labels et légendes des graphiques -> Chaque courbe tracée dispose d'une couleur distincte et contrastée. La légende s'actualise dynamiquement pour indiquer le nom de l'entreprise et la pastille de couleur associée.

US #7 — Exportation en PNG -> Un bouton dédié permet d'enregistrer le graphique actif sous forme d'image au format .png via une boîte de dialogue standard pour choisir le dossier de destination.

Voici les user stories ou certaines parties d'entre elles qui ont été abandonnés à cause de manque de temps ou qui avaient peu de valeur, tels que :

US #1 — Importation directe de fichiers .JSON externe -> J'ai décidé de ne pas faire un parser pour les fichier .json, parce que .csv serait le format le plus utilisé pour ce genre de données.

US #5 — Importation en direct via une API externe -> cette fonctionnalité a été complétement abandonnée dès le début, à cause de sa complexité.

US #6 — Saisie manuelle d'un nouveau point -> cette user story a été abandonné parce qu'elle n'apporte aucune valeur ajoutée pour l'utilisateur, et qu'elle n'était pas obligatoire dans le cahier des charges.

7.3 Bilan des problèmes

- C'était problématique de trouver une bonne source pour les séries de données, et j'ai dû passer quelques périodes à trouver une solution.

- C'était aussi un peu compliqué à comprendre comment fonctionne WinForms, lors de placement des éléments (boutons, sections, etc) sur la fenêtre et de gérer les différentes résolutions d'écran.

7.4 Bilan du projet

Ce projet m'a permis de monter mes compétences en C#, particulièrement dans l'usage de LINQ et la mise en place de tests unitaires avec MSTest. L'intégration de ScottPlot a été effectué et m'a permis d'obtenir de l'expérience avec ce genre de librairies.

Comme suites possibles, l'application pourrait intégrer des indicateurs boursiers complémentaires (moyennes mobiles) et un sélecteur de couleurs pour personnaliser chaque courbe, et pouvoir choisir des différentes courbes.

8 WEBOGRAPHIE

Scottplot – Guide officiel de ScottPlot pour WinForms - https://scottplot.net/cookbook/5.0/

Google Gemini – explications et suggestions par rapport au code - gemini.google.com

Yahoo Finance 2 – bibliothèque de scraping de données historiques de la bourse - https://github.com/gadicc/yahoo-finance2/tree/dev